LAB 8: Building an Interactive Dynamic Service Registry & Theme Dashboard with Intermediate JavaScript

Lab Objective

In this lab, you will apply the core intermediate JavaScript concepts introduced in Lecture 8 including addEventListener(), classList manipulation, JavaScript **Arrays**, and dynamic DOM rendering via .forEach(), .createElement(), and .appendChild() to build an interactive, single-page **System Service Registry & Theme Dashboard**.

Working without starter code templates, you will construct the HTML structure, style it using Bootstrap 5 classes, and write clean JavaScript to manage stateful theme toggling and render dynamic component lists from data arrays.

Project Requirements & Open-Ended Scenario

You have been hired by an enterprise infrastructure team to create a management dashboard prototype. The dashboard needs to feature two interactive panels:

1. **Theme Switcher Panel (Advanced ClassList & Event Listeners):** An interactive UI box that allows users to toggle between Light and Dark display modes cleanly using professional addEventListener() bindings and classList.toggle().
2. **Dynamic Service Registry (Arrays & DOM Injection):** A data-driven module list that stores system services in an array and automatically loops through them using .forEach() to render interactive list items with status badges into the DOM.

Rubric Breakdown (100 Points Total)

Your lab submission will be evaluated against the following itemized criteria:

Criteria	Description	Points
1. File Setup & Script Linkage	HTML boilerplate linked to Bootstrap 5 CDN via <head>, containing a single <script> block placed right before the closing </body> tag. Zero console errors on page load.	**15 pts**
2. Variable Declarations & Scope	Correct, explicit use of modern ES6 keywords (const for DOM element selections, arrays,	**15 pts**

Criteria	Description	Points
	and fixed values; let where applicable). Zero usage of legacy var.	
3. Modern Event Handling (addEventListener)	Clean separation of concerns by binding interactive buttons using addEventListener() instead of inline onclick attributes.	**25 pts**
4. Advanced ClassList & Dynamic Array Rendering	Proper use of classList.toggle() and classList.contains() for UI styling switches. Successful use of .forEach(), .createElement(), and .appendChild() to render array data into the DOM.	**25 pts**
5. Code Structure, Formatting & Comments	Clean, readable code structure with clear function definitions, consistent indentation, meaningful variable names, and clear comments explaining key DOM operations.	**20 pts**

Detailed Implementation Guide

Part 1: Page Layout & HTML Structure (index.html)

Build a single-page HTML dashboard structured with Bootstrap 5 container grid classes.

1. **Header Section:** A navigation bar or header displaying "DevPortal | Intermediate JS Lab".
2. **Card 1: Theme Switcher Card:**
    - A styled text box container with an id="theme-box".
    - A toggle button with an id="toggle-btn".
3. **Card 2: Dynamic Service Registry Card:**
    - A card header displaying "Active System Modules".

- An empty unordered list container with an id="module-list" and the Bootstrap class list-group.

Part 2: JavaScript Implementation Instructions

Inside your <script> tag at the bottom of index.html, implement the following functional requirements:

A. Modern Event Listener Logic

- Select the toggle button and theme box using document.querySelector().
- Attach an addEventListener("click", ...) handler to manage class toggling (bg-dark, text-white, border-dark) and button text updates safely.

B. Array Definition & Console Diagnostics

- Declare an array named systemModules containing at least 4 string names (e.g., "Authentication Engine", "Database Connector", "Cloud Storage API", "Logging Service").
- Output diagnostic information to the console using console.log() checking array length, the first item ([0]), and the last item ([length - 1]).

C. Dynamic DOM Rendering Logic

- Select the list container (#module-list) using document.querySelector().
- Use .forEach() to iterate through the systemModules array.
- Inside the loop, create new <li> elements using document.createElement(), apply Bootstrap layout classes, inject structured HTML via template literals containing badges, and append them using .appendChild().

Submission Instructions

1. Save your completed code as an index.html file in your project directory.
2. Open index.html in your web browser and test the theme toggle button and verify that all four system modules render cleanly inside the list group.
3. Open Developer Tools (F12 or Right-Click -> Inspect -> Console) to verify console log outputs and confirm there are zero syntax or reference errors on load.